# FusedLinearAttention Final Report

## 1. Overview

This report summarizes the final state of the FusedLinearAttention project:

- what was built
- what experiments were run
- how the fused kernel compares with the PatchTST baseline
- what optimizations were attempted
- what helped, what did not, and why the fused kernel still trails baseline

The canonical result files referenced in this report live in:

- baseline_pipeline/results

The final H100 performance comparison is captured in:

- comparison_table.csv
- validation_table.csv
- endtoend_timing.csv
- correctness_results.csv

## 2. Objective

Standard transformer attention does:

1. `X -> Q, K, V` through three separate projections
2. scaled dot-product attention over the projected tensors

This causes:

- multiple kernel launches
- intermediate `Q/K/V` writes to HBM
- re-reads of those intermediates during attention

The goal of this project was to fuse these stages into a single CUDA kernel to:

- reduce kernel launch count
- reduce HBM traffic
- improve throughput on GPU inference

# 3. Full Pipeline Delivered

## Baseline

- ETTh1 preprocessing and DataLoaders
- PatchTST baseline model
- training and evaluation scripts
- baseline profiling script

## Fused path

- canonical CUDA kernel in kernel/fused_attn.cu
- C++ extension binding in kernel/fused_attn_ext.cpp
- JIT loader in kernel/load_kernel.py
- model-side fused wrapper in baseline_pipeline/model/fused_attn_block.py
- fused profiling and result-merge workflow
- end-to-end fused validation / retraining workflow

## Verification

- NumPy / PyTorch reference workflow
- CUDA correctness sweep
- generated figures and summary tables

# 4. Experimental Setup

## Model-side benchmark configuration

From baseline_pipeline/config.py:

- benchmark embed dim: `512`
- benchmark heads: `8`
- head dim: `64`

- batch size: `1`
- sequence lengths: `64, 128, 256, 512, 1024`
- warmup iterations: `100`
- timed iterations: `500`

### End-to-end model configuration

- dataset: ETTh1
- model: PatchTST
- `d_model=128`
- `n_heads=4`
- `n_layers=2`
- `batch_size=32`

### Final GPU environment

The final compiled-kernel runs were executed on an NVIDIA H100 with:

- CUDA toolkit available on host
- PyTorch CUDA build
- `TORCH_CUDA_ARCH_LIST=9.0`

## 5. Final Baseline vs Fused Results

## 5.1 Forward microbenchmark

Source: comparison_table.csv

| Seq Len | Baseline ( us ) | Fused ( us ) | Fused Speed vs Baseline | HBM Read Reduction |
|---|---|---|---|---|
| 64 | 109.7448 | 872.0335 | 0.1258x | 10.71% |
| 128 | 109.1533 | 1422.1433 | 0.0768x | 18.75% |
| 256 | 108.9229 | 2532.7407 | 0.0430x | 30.00% |
| 512 | 118.5030 | 4737.3379 | 0.0250x | 42.86% |
| 1024 | 198.5365 | 9124.8203 | 0.0218x | 54.55% |

### Interpretation

The fused kernel achieves the intended memory-traffic trend:

- estimated HBM reads decrease relative to baseline
- the reduction becomes more pronounced at longer sequence lengths

However, the kernel still loses badly on wall-clock time. The current kernel reduces memory traffic but does not execute the projection math efficiently enough to beat the PyTorch baseline.

## 5.2 End-to-end timing

Source: endtoend_timing.csv

| Model | Mean Epoch Time ( s ) | Forward per Iter ( ms ) | Speed vs Baseline |
|---|---|---|---|
| Baseline | 1.8825 | 0.544459 | reference |
| Fused | 2.5510 | 0.633303 | 0.859713x forward, 0.737946x epoch |

### Interpretation

The fused path remains slower even in the end-to-end pipeline. The forward path is slower, and because training uses a recomputation-based backward bridge, the full epoch time is slower as well.

## 5.3 Forecast metrics

Source: validation_table.csv

| Model | MSE | MAE |
|---|---|---|
| Baseline | 180.46336365 | 12.64666843 |
| Fused | 213.44288635 | 13.50915241 |

Delta vs baseline:

- MSE: `+18.274913%`
- MAE: `+6.81985129%`

- within 1% target: `NO`

### Interpretation

The fused model path is currently less accurate than baseline in the final PatchTST experiment. Even though the kernel itself is numerically correct on standalone attention tests, the final end-to-end training outcome still lags.

## 5.4 Correctness

Source: correctness_results.csv

Result:

- `11 / 11` CUDA correctness cases passed
- max abs diff remained on the order of `1e−7`

### Interpretation

This is an important distinction:

- the fused kernel is **correct**
- the fused kernel is **not yet fast enough**

So the primary remaining problem is performance architecture, not numerical correctness.

## 6. Figures

The generated figures live under baseline_pipeline/results/figures.

Recommended figures for presentation:

- speedup.png
- direct fused-vs-baseline speed comparison
- wall_time_speedup.png
- wall-time comparison view
- hbm_bandwidth.png
- memory-traffic/bandwidth-oriented comparison
- kernel_count.png

- fused launch count vs baseline
- occupancy_vs_tile.png
- occupancy/tile-size sweep
- occupancy_vs_tile_tradeoff.png
- tuning tradeoff visualization
- nsight_timeline_comparison.png
- timeline-style comparison artifact

## 7. Optimization Attempts and Outcomes

This section records the main kernel optimization passes tried during the project and what happened.

### Attempt 1: More parallel launch structure

Change:

- moved from one block per `(B, H)` to one block per `(B, H, q_tile)`

Why:

- the original launch strategy exposed far too little parallelism on H100

Outcome:

- large improvement
- this was one of the highest-return fixes

### Attempt 2: Architecture-aware compilation

Change:

- removed the hardcoded `sm_80`
- compiled for the actual active GPU architecture

Why:

- the final experiments ran on H100, which needs `sm_90`

Outcome:

- necessary correctness/performance hygiene

- helped avoid leaving hardware-specific performance on the table

### Attempt 3: Dynamic shared memory and larger tiles

Change:

- moved from static shared-memory layout to dynamic shared memory
- enabled larger tile sizes again

Why:

- earlier versions hit shared-memory size limits

Outcome:

- improved the viable tile strategy
- tile size `64` ended up clearly outperforming `32` and `16`

### Attempt 4: Projection tiling through shared memory

Change:

- staged input chunks and weight chunks into shared memory before projection

Why:

- projection loops were clearly the dominant bottleneck

Outcome:

- this was the biggest performance win after the launch fix
- it reduced the fused forward time by an order of magnitude versus the oldest committed fused-kernel results

### Attempt 5: Two-pass softmax to reduce per-thread score storage

Change:

- removed per-thread `scores[]`
- recomputed scores in a second pass

Why:

- intended to reduce register pressure

Outcome:

- performance got worse
- extra dot-product recomputation cost more than the register savings
- reverted

### Attempt 6: Larger projection staging tile

Change:

- swept `PROJ_K_TILE` upward

Why:

- looked like a possible way to increase arithmetic intensity

Outcome:

- worse than the smaller setting
- `PROJ_K_TILE=8` remained best

### Attempt 7: Register cap sweep

Change:

- tested `-maxrregcount` settings

Why:

- tried to trade registers for occupancy

Outcome:

- every forced cap was worse than the unconstrained compiler choice
- reverted

### Attempt 8: Cooperative query-group rewrite

Change:

- split each query row across multiple threads

Why:

- intended to reduce per-thread vector ownership and register pressure

Outcome:

- preserved correctness
- performance regressed due to additional synchronization and score staging
- reverted

### Attempt 9: Launch-bounds hint

Change:

- added a small `__launch_bounds__(TILE_SIZE, 2)` hint

Why:

- light-touch compiler guidance without changing the algorithm

Outcome:

- very small improvement
- kept in the final kernel because it helped slightly and did not hurt

## 8. Why the Fused Kernel Is Still Slower

The central result is:

- the fused kernel **does reduce estimated HBM traffic**
- but it still **does not beat** the PyTorch baseline on wall-clock time

The most likely reasons are:

1. **Scalar fp32 projection loops**
2. Q/K/V projection is still done with scalar multiply-add loops
3. PyTorch baseline likely hits highly optimized kernels underneath
4. **No tensor-core usage**
5. the implementation is not structured around MMA/tensor-core execution
6. H100 is especially strong when kernels are built for tensor cores

7. **Projection cost dominates**
8. fusion removed some memory movement
9. but the saved memory traffic is outweighed by compute inefficiency
10. **Training path is not fully fused**
11. backward uses a custom autograd recomputation bridge
12. so epoch time is not a pure fused-forward-plus-fused-backward measurement

# 9. Final Takeaways

### What succeeded

- the project delivered a working fused CUDA kernel path
- the kernel passed standalone correctness checks
- the full profiling and validation pipeline is complete
- major kernel regressions were removed
- the fused forward path became much faster than early versions

### What did not succeed

- the fused kernel did not beat the PyTorch baseline
- the final fused PatchTST model underperformed baseline on MSE/MAE

### The honest conclusion

This project successfully demonstrates:

- how to fuse projection and attention into one kernel
- how to validate and benchmark it rigorously
- how memory-traffic reduction alone is not enough to guarantee better runtime

The current kernel is a solid correctness-and-systems result, but it is not yet the final high-performance design needed to outperform optimized library attention on H100.

# 10. Recommended Next Steps

If the project were extended, the highest-return next steps would be:

1. rewrite projection around tensor-core-friendly tiles
2. move to bf16/fp16 compute
3. redesign warp-level work partitioning
4. implement a real fused CUDA backward kernel
5. collect Nsight Compute occupancy and instruction-level diagnostics for the best current kernel

## 11. Canonical Deliverables

For convenience, the final project artifacts are:

- README.md
- comparison_table.csv
- validation_table.csv
- endtoend_timing.csv
- correctness_results.csv
- figures/